Reviewer Pack — Minimal Index of Symplectic Leaves Bounds DPLL Path Length

Entry cdb7a257ee09 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 15:02:12 UTC

Minimal Index of Symplectic Leaves Bounds DPLL Path Length

Entry ID: `cdb7a257ee09` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 15:02:12 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): DPLL Search Trees in Complexity Theory

Statement:

> For every CNF formula φ, the minimal index of symplectic leaves (misl(φ)) of its associated toric variety is linearly correlated with its DPLL path length, such that misl(φ) = Θ(w(φ)), where w(φ) is the DPLL path length of φ.

Rationale (proposer's reasoning):

> Symplectic geometry provides a geometric interpretation for quantum entanglement and complexity measures. The minimal index of symplectic leaves could potentially capture non-trivial geometric information that correlates with the hardness of SAT instances, as reflected in their DPLL path lengths.

Taxonomy category: `symplectic_geometry_dpll` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `81a449852fc0f16d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture that minimal index of symplectic leaves (misl(φ)) is linearly correlated with DPLL path length (w(φ)), support will be considered if misl(φ) = Θ(w(φ)) AND correlation coefficient r >= 0.8, for at least 24 out of 30 random seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `symplectic geometry AND DPLL path length` - `toric variety AND minimal index of symplectic leaves` - `DPLL search trees IN complexity theory AND symplectic geometry`

Top relevant hits considered: - [http://arxiv.org/abs/math/0601540v1] Symplectic forms and surfaces of negative square - [http://arxiv.org/abs/1811.09984v4] Translated points for contactomorphisms of prequantization spaces over monotone symplectic toric manifolds - [http://arxiv.org/abs/1007.4036v2] Symplectic reduction of quasi-morphisms and quasi-states - [http://arxiv.org/abs/1906.02237v2] ECH capacities, Ehrhart theory, and toric varieties - [http://arxiv.org/abs/dg-ga/9511008v1] Hamiltonian torus actions on symplectic orbifolds and toric varieties - [http://arxiv.org/abs/math/0004122v1] Kahler geometry of toric manifolds in symplectic coordinates - [http://arxiv.org/abs/math/0308183v3] Compactness results in Symplectic Field Theory - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

# Define a small DPLL solver
def dpll(clause_set, assignment, clauses):
    if not clause_set:
        return True
    unit_clause = next((c for c in clause_set if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        if literal < 0:
            literal = -literal
        assignment[literal - 1] = True
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        return dpll(new_clauses, assignment, clause_set)
    pure_literal = next((lit for lit in range(1, len(assignment) + 1) if (lit not in assignment ar
    if pure_literal is None:
        return False
    literal = pure_literal
    assignment[literal - 1] = True
    new_clauses = [c for c in clauses if literal not in c and -literal not in c]
    if dpll(new_clauses, assignment, clause_set):
        return True
    assignment[literal - 1] = False
    new_clauses = [c for c in clauses if literal not in c and -literal not in c]
```

```
    if dpll(new_clauses, assignment, clause_set):
        return True
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random CNF formula of size n
    n = 10 + random.randint(0, 29)
    m = 5 * n + random.randint(0, n)
    clause_set = []
    for _ in range(m):
        num_literals = random.randint(1, n)
        literals = random.sample(range(1, n + 1), num_literals)
        clause = [l if random.choice([True, False]) else -l for l in literals]
        clause_set.append(clause)

    # Measure DPLL path length
    assignment = [None] * n
    result = dpll(clause_set, assignment, clause_set)
    w_phi = len(assignment)  # Simplified DPLL path length

    # Compute minimal index of symplectic leaves (misl(φ))
    # This is a placeholder function. Replace with actual computation.
    misl_phi = n  # Placeholder value

    return {
        "metric_name": "misl_phi",
        "metric_value": misl_phi,
        "instances_tested": m,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_misl_phi = sum(r["metric_value"] for r in results) / len(results)
    std_misl_phi = math.sqrt(sum((r["metric_value"] - mean_misl_phi) ** 2 for r in results) / len(
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_misl_phi} std={std_misl_phi} support_fraction={suppor
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b169495.py", line 80, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b169495.py", line 59, in run_trial
    result = dpll(clause_set, assignment, clause_set)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b169495.py", line 29, in dpll
    return dpll(new_clauses, assignment, clause_set)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b169495.py", line 29, in dpll
    return dpll(new_clauses, assignment, clause_set)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b169495.py", line 29, in dpll
    return dpll(new_clauses, assignment, clause_set)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 994 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b169495.py", line 22, in dpll
    unit_clause = next((c for c in clause_set if len(c) == 1), None)
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide the required correlation coefficient and number of seeds with correct correlation. | next: Re-run the test without errors to verify if misl(φ) is linearly correlated with DPLL path length (w(φ)) as per the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14996
2	preregistration	ollama_remote	glm4:latest	0	0	9664
3	novelty	ollama_remote	glm4:latest	0	0	8230

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	10222
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20541
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17468
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24860
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25261
9	judge	ollama_remote	glm4:latest	0	0	12289

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 143531 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cdb7a257ee09.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cdb7a257ee09.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cdb7a257ee09.tar.gz` (if generated)